Assignment 2 (Weeks 3 and 4)

PART 1

1.

The query always returns zero because NULL does not represent any value and hence the where condition is never satisfied.

The correct query

```
SELECT COUNT(*)
FROM Student
WHERE phone_number IS NULL;
```

NULL is excluded from most aggregate/ comparison operators because NULL represents unknown value and an unknown cannot be compared and no operations can be performed on it either.

2.

Yes, the distinct Keyword here is redundant. The distinct keyword is used to remove all the duplicate values. In the given query we are trying to identify how many employees are there in each department. Using a distinct keyword would make sure that a department appears only once in the output but this is redundant because group by function already does this task by grouping all the records that have the same department.

3.

When using the left join, we take all the rows from the left table and all the matching rows from the right table. Whereas the inner join only returns the matching rows from both the tables.

The difference in count of records in left join and inner join indicates that there exist rows in the left table that don’t have any matching record in the right table.

4.

Given query :

```
SELECT department, employee_name, AVG(salary)
FROM Employee
GROUP BY department;
```

The given query is wrong because by SQL syntax rules, all the non-aggregate fields which appear in the SELECT clause have to be used in group by function as well.

Corrected query :

```
SELECT department, employee_name, AVG(salary)
FROM Employee
GROUP BY department, employee_name;
```

If this were conceptually allowed, the sql query would first group all the records by the departments, then calculate average salary for each department but when it would come to displaying employee_name in the output, it would be confused as to which employees name should be displayed since each department group contains several employees. So if the given query were allowed it would arbitrarily choose any employee from the grouping and display its name or maybe it would display the first employee name that appears in each group.

5.

The given query fails because the where condition uses avg salary which is not calculated yet and even if this were allowed, the where condition would filter the rows before the grouping happens which would remove all the records with avg salary less than 80,000 and the avg salary per department would not represent the avg salary of the department since some records were not considered.

Corrected query :

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000 ;
```

6.

a) Non-correlated subquery is better since we don't need any information from the outer query to execute the inner query.
b) Correlated subquery is better since we need department information from the outer query to execute the inner query.

A non-correlated subquery is usually executed once, whereas a correlated subquery may execute once for every row of the outer query. Therefore, correlated subqueries can be much slower when the outer table is large.

PART 2

A.

```
SELECT c.customer_id, c.name
FROM Customer as c
LEFT JOIN order as o
ON  c.customer_id = o.customer_id
WHERE o.customer_id IS NULL ;
```

I have used an exclusive left join to isolate the customers who have never placed an order.

B.

```
SELECT
   r.restaurant_id,
   r.name,
   AVG(a.order_value) AS avg_order_value
FROM
   Restaurant AS r
   INNER JOIN
   ( SELECT
         o.restaurant_id,
         o.order_id,
```

```
        SUM(oi.quantity*mi.price) AS order_value
      FROM
        Order as o
        INNER JOIN OrderItem as oi ON o.order_id = oi.order_id
        INNER JOIN MenuItem as mi ON oi.item_id = mi.item_id
      GROUP BY o.restaurant_id, o.order_id ) a
    ON r.restaurant_id = a.restaurant_id
    GROUP BY r.restaurant_id, r.name
    HAVING COUNT(*) > 5 ;
```

Since we had to use information from many tables, I used a subquery to create a new table which reduced the problem into 2 tables. The Having statement is used to ensure that the number of records (in another sense the number of orders) is greater than 5.

C.

```
SELECT res.restaurant_id, res.name
FROM
( SELECT
r.restaurant_id,
r.name,
AVG(a.order_value) AS avg_order_value
FROM
Restaurant AS r
INNER JOIN
( SELECT
o.restaurant_id,
o.order_id,
SUM(oi.quantity*mi.price) AS order_value
FROM
Order as o
INNER JOIN OrderItem as oi ON o.order_id = oi.order_id
INNER JOIN MenuItem as mi ON oi.item_id = mi.item_id
GROUP BY o.restaurant_id, o.order_id ) a
ON r.restaurant_id = a.restaurant_id
GROUP BY r.restaurant_id, r.name
```

```
HAVING COUNT(*) > 5 ) as res
Where res.avg_order_value >
( SELECT AVG(p.p_order_value)
FROM
(SELECT
SUM(oi.quantity*mi.price) AS p_order_value
FROM
Order as o
INNER JOIN OrderItem as oi ON o.order_id = oi.order_id
INNER JOIN MenuItem as mi ON oi.item_id = mi.item_id
GROUP BY o.order_id) p ) ;
```

We don't require a correlated subquery here since the platform avg order value is calculated once and doesn't require any information from the outer query.

D.

```
SELECT t.restaurant_id, t.item_name
FROM (
    SELECT
        o.restaurant_id,
        mi.item_id,
        mi.item_name,
        SUM(oi.quantity) AS total_quantity
    FROM `Order` o
    JOIN OrderItem oi
        ON o.order_id = oi.order_id
    JOIN MenuItem mi
        ON oi.item_id = mi.item_id
    GROUP BY
        o.restaurant_id,
        mi.item_id,
        mi.item_name
) t
WHERE t.total_quantity = (
    SELECT MAX(t2.total_quantity)
    FROM (
        SELECT
            o.restaurant_id,
            mi.item_id,
```

```
        SUM(oi.quantity) AS total_quantity
    FROM `Order` o
    JOIN OrderItem oi
        ON o.order_id = oi.order_id
    JOIN MenuItem mi
        ON oi.item_id = mi.item_id
    GROUP BY
        o.restaurant_id,
        mi.item_id
  ) t2
  WHERE t2.restaurant_id = t.restaurant_id
);
```

Group by with max returns only the maximum value, not the row associated with that value. To retrieve the corresponding menu item (or handle ties), additional joins, subqueries, or window functions are required

PART 3

1. The functional dependencies are :
   student_id → student_name
   course_id → course_name
   instructor_id → instructor_name
   course_id → instructor_id
   instructor_id → instructor_office
   { student_id, course_id } → grade
   { student_id, course_id } → enrollment_date

   The multivalued dependencies are
   student_id —->> course_id
   course_id —->> student_id

2. Assuming { student_id, course_id } is the primary key
   yes , the table is in 1NF because
   - There is a primary key
   - All columns allow a single datatype
   - There are no groups in any column
   - The row order is not relevant
3. There are several partial dependencies like :
   - student_name depends only on student_id

- Course_name depends only on course_id
- Instructor_id depends only on course_id

Insertion anomaly : a course which does not have any student registered cannot be inserted into the table

Updation anomaly : a course has several students but only one instructor. So in a case where the instructor of a course changes, we have to update several records.

Deletion anomaly : in a case where a student is not registered in any course in the future when we delete this student's record from the table, we lose information about the student like their name.

3. The transitive dependencies are :
   - course_id → instructor_id → instructor_name
   - course_id → instructor_id → instructor_office

   The anomaly it causes is when change the instructor name changes, we have to update the course_id and instructor_id for several records and we can miss updating some which will cause data inconsistency

4. Student (student_id, student_name)
   Stu_course(student_id, course_id, grade, enrollment_date)
   Course (course_id, course_name, instructor_id)
   Instructor(instructor_id, instructor_name, instructor_office)

   The Student table justifies the functional dependency :
   student_id → student_name

   The Stu_course table justifies the functional dependency :
   { student_id, course_id } → grade
   { student_id, course_id } → enrollment_date

   The Course table justifies the functional dependency :
   course_id → course_name
   course_id → instructor_id

   The Instructor table justifies the functional dependency :
   instructor_id → instructor_office

instructor_id → instructor_name

PART 4

For Part II (a), which required listing all customers who have never placed an order, the query could have been written using either a LEFT JOIN or a subquery with NOT IN.

An alternative query is:

```
SELECT customer_id, name

FROM Customer

WHERE customer_id NOT IN (

   SELECT customer_id

   FROM `Order`

);
```

I chose the LEFT JOIN approach because it is more readable and clearly shows that all customers are included first, and then only those without matching orders are selected by checking for NULL values. It directly represents the relationship between the Customer and Order tables, making the query easier to understand.

The LEFT JOIN approach is also safer when handling NULL values. If the subquery used with NOT IN returns a NULL value, the comparison may not behave as expected and can return no rows. Using LEFT JOIN with `WHERE order_id IS NULL` avoids this issue and correctly identifies customers who have never placed an order.

From a performance perspective, modern database systems generally optimize both approaches efficiently, so the choice was primarily based on readability and reliable handling of NULL values

2. Normalizing the Enrollment table into 3NF removes redundancy and improves data integrity, but it also makes some queries less convenient. After decomposition, information about students, courses, instructors, and enrollments is stored in separate tables. As a result, queries that previously accessed a single table must now use multiple JOIN operations to retrieve complete information, making them longer and more complex.

For example, displaying a student's name, course name, instructor name, and grade now requires joining the Student, Course, Instructor, and Stu_Course tables. This can slightly increase query complexity and, for large databases, may also increase query execution time.

In practice, some systems intentionally keep a small amount of denormalization to improve performance. Frequently accessed data, such as instructor names or course names, may be duplicated in another table to reduce the number of JOINs required. Although this introduces some redundancy and requires extra care when updating data, it can significantly improve the performance of read-heavy applications such as reporting systems or dashboards. Therefore, database designers often balance normalization and denormalization based on the system's performance and maintenance requirements.